Machine Learning & Deep Learning

UniVR - Dipartimento di Informatica

Tommaso Toffali

1° Semestre 2026/2027

Indice

Parte I
Introduction to Machine Learning

1 What is Machine Learning?

Machine Learning is a branch of Artificial Intelligence that enables software to use data to find solutions to specific tasks without being explicitly programmed to do so.

1.1 Machine Learning vs Statistical Modelling

In traditional statistical modelling, we follow a structured process:

1. Collect data
2. Verify and clean the data (correct or discard if not clean)
3. Use the clean data to test hypotheses, make predictions and forecasts

In contrast, Machine Learning takes a different approach:

- It is the **data** that determines which analytic techniques to use
- The computer uses data to **train** algorithms to find patterns and make predictions
- Algorithms are no longer **static** but become **dynamic**

1.2 Conventional Programming vs Machine Learning

The fundamental difference between conventional programming and machine learning can be undersood through their inputs and outputs:

1.2.1 Conventional programming

- **Input**: Program + Data
- **Output**: Result
- You write the program, give it data, and get results

1.2.2 Machine Learning

- **Input**: Data + Result
- **Output**: Result
- You give the computer data and desired results, and it learns to generate a program/algorithm

1.3 The Machine Learning Recipe

Definizione 1.1 (Fundamental Steps of ML)

Collect data → Define models → Define objective function → Minimize error

The ML recipe consists of 4 fundamental step:

1. Collect data
2. Define a family of possible models
3. Define an objective (error) function to quantify how well a model fits the data
4. Find the model that minimize the error function (training/learning a model)

1.3.1 The Key ingredients

Esempio 1.2 (Essential Ingredient)

Task • Data • Model • Objective Function • Learning Algorithm

Every machine learning problem requires five essential ingredients:

- **Task**: The problem we want to solve
- **Data**: The information we use to learn
- **Model hypothesis**: The family of functions we consider
- **Objective function**: How we measure success
- **Learning algorithm**: How we fine the best model

2 Machine Learning Tasks

A task represent the type of prediction being made to solve a problem on some data. We can identify a task with the set of functions that can potentially solve the problem. In generale, it consists of functions assigning each **input** $x \in \mathcal{X}$ an **output** $y \in \mathcal{Y}$.

2.1 Classification task

Definizione 2.1 (Classification)

Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ assigning each input $x \in \mathcal{X}$ a **discrete** label, $f(x) \in \mathcal{Y} = \{c_1, \ldots, c_k\}$.

In classification, the output space is a finite set of discrete categories or classes. The goal is to learn a decision boundary that separates different classes.

2.2 Regression task

Definizione 2.2 (Regression)

Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ assigning each input $x \in \mathcal{X}$ a **continuos** label, $f(x) \in \mathcal{Y} = \mathbb{R}$.

In regression, the output is a continuos value, allowing for predictions of quantities such as prices, temperatures, or probabilities.

2.3 Density Estimation task

Definizione 2.3 (Density Estimation)

Find a probability distribuition $f \in \Delta(\mathcal{X})$ that best fits the data $x \in \mathcal{X}$.

There is no reference to an output space (as in calssification and regression task). Instead, we make a reasoning on the **input** x itself, trying to model the underlying probability distribution that generated the data.

2.4 Clustering task

Definizione 2.4 (Clustering)

Find a function $f \in \mathbb{N}^{\mathcal{X}}$ that assigns each input $x \in \mathcal{X}$ a **cluster index** $f(x) \in \mathbb{N}$.

All points mapped to the same index form a cluster. The goal is to group similar data points together without prior knowledge of the groups.

2.5 Dimensionality Reduction Task

Definizione 2.5 (Dimensionality Reduction)

Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ that **maps** each (**high-dimensional**) input $x \in \mathcal{X}$ to a **lower-dimensional** embedding $f(x) \in \mathcal{Y}$, where $dim(\mathcal{Y}) < dim(\mathcal{X})$.

3 Data in Machine Learning

3.1 Data representation

We represent inputs of out algorithm as $x \in \mathcal{X}$ and output as $y \in \mathcal{Y}$. The dataset is the set of both, where each input is associated with an output.

Key consideration:

- The quality of the data is crucial for the performance of the algorithm
- It is equally important to have a large amount of data to learn effective models
- Poor quality or insufficient data can lead to poor model performance

3.2 Features

Definizione 3.1 (Features)

Features are the measurable properties or charateristics of the phenomenon being observed

Examples:

- To identify a flower: color, petal length, petal width, sepal length, sepal width
- To identify a fruit: color, shape, size, texture, weight
- To classify images: pixel values, edges, textures, shapes

We can say that features are how algorithm "sees" the data and are in general represented with (fixed-size) vectors.

4 Data Distributions and Learning Types

In machine learning, the information about the problem we want to solve is represented in the form of a data distribution, usually denoted as p_{data}.

4.1 Supervised learning

4.1.1 Classification and Regression

The data distribution is over pairs of inputs and outputs:

$$p_{data} \in \Delta(\mathcal{X} \times \mathcal{Y})$$

Here:

- $\mathcal{X}$ is the input space
- $\mathcal{Y}$ is the output space (labels or targets)
- The goal is to learn a mapping from inputs to outputs using labeled data

4.2 Unsupervised Learning

4.2.1 Density Estimation, Clustering and Dimensionality reduction

The data distribution is only over the input space:

$$p_{data} \in \Delta(\mathcal{X})$$

Here:

- We do not have explicit labels or targets
- The goal is to discover structure, patterns or representations within the data itself

4.3 Summary of Learning Types

Task type	Data distribution	Learning Type
Classification, Regression	$p_{data} \in \Delta(\mathcal{X} \times \mathcal{Y})$	Supervised Learning
Density Estimation, Clustering, Dimensionality Reduction	$p_{data} \in \Delta(\mathcal{X})$	Unsupervised Learning

Note. In **supervised learning**, each data point consistes of an input and corresponding output (label). In **unsupervised learning**, each data point consists only of the input, and the algorithm tries to find pattersn or structure without explicit labels.

The distinction is fundamental in machine learning, as it determines the type of algorithms and approaches used to solve different problems.

5 Data sampling and dataset splitting

5.1 The data distribution problem

In both supervised and unsupervised learning, the true data distribution p_{data} is typically **unknown**. This presents a fundamental challenge:

- We do not have direct access to the underlying probability distribution that generates data
- We only have access to a finite sample of data points drawn from this distribution

Solution: We perform **sampling** from the distribution. By collecting a representative sample of data, we can approximate the true distribution and use it to train our models.

5.2 Training, Validation and Test sets

To properly evaluate machine learning models, we split out data into three distinct sets. Each set serves a specific purpose inthe machine learning pipeline.

Definizione 5.1 (Dataset Splitting)

A dataset is typically divided into three subsets:

- **Training set** (D_{train}): Used to train the model
- **Validation set** (D_{val}): Used to tune hyperparameters and select the best model
- **Test set** (D_{test}): Used to evaluate the final model performance

Definizione 5.2 (Critical Requirement)

All three sets (training, validation, test) must be sampled from the **same probaility distribution**

5.3 How to use the three sets

The three datasets serve different purposes in the machine learning workflow:

5.3.1 Learning/Training Phase

During the learning pahse, the **training set** is used to train the machine learning algorithm, which learns patterns and realtionships in the data to produce a **program** (model)

The **validation set** plays a crucial role in this phase:

- Evaluates different model configurations
- Tunes hyperparameters
- Selects the best performing model

This process involves iterative feedback between training and validation until we find the optimal configuration.

5.3.2 Testing/Model evaluation Phase

Once the best model is selected, we use the **test set** to evaluate its performance. The trained model is applied to the test data, which it has never seen before. This gives us an unbiased estimated of how well the model will perform in real-world scenarios, telling us whether out model can truly generalize to new data.

5.4 The importance of Distribution similarity

5.4.1 Correct Scenario: Same Distribution

When training, validation, and test sets are sampled from the **same distribution**:

- The sets are **similar** in terms of statistical properties
- They represents the same underlying phenomenon
- However, they **do not overlap** (no data point appears in multiple sets)

This ensures that the model cna generalize well from training to test data.

Esempio 5.3 (Fruits)

If we're classifying fruits, all three sets might contain apples and bananas in similar proportions, but with different specific instances.

5.4.2 Incorrect Scenario: Different Distribution

When training/validation and test come from **different distributions**, the model learns patterns from one distribution but is tested on another. This problematic scenario typically leads to:

- Poor performance on the test set

- Unrealiable predictions on new data
- Inability to generalize to real-world applications

Esempio 5.4 (Different distributions)

Training on apples and bananas, but testing on oranges and limes. The model cannot calssify fruits it has never seen during training

Teorema 5.5 (Key principle)

Training and test data must come from the **same distribution**, violating thi leads to poor generalization

6 Training and Testing: The role of features

6.1 Training (Learning, Induction)

During the training phase, the model learns to distinguish between different classes based on the features provided in the training data.

Esempio 6.1 (Fruit classification)

Consider a fruit classification task with the following training data:

- *red, round, leaf, 3oz, ...* $\rightarrow$ apple
- *green, round, no leaf, 4oz, ...* $\rightarrow$ apple
- *yellow, curved, no leaf, 8oz, ...* $\rightarrow$ banana
- *green, curved, no leaf, 7oz, ...* $\rightarrow$ banana

The model learns to associate patterns in the features (color, shape, presence of leaf, weight) with the corresponding labels (apple or banana). **What distinguishes apples and bananas is based on the features!** The model identifies which feature combinations are characteristic of each class.

6.2 Testing

Once trained, the model can classify new examples based on the same features it learned during training. When presented with a new example described byt the same semantic features, the model make a prediction

Esempio 6.2 (Fruit)

Given a new fruit with features *red, round, no leaf, 4oz, ...*, the model uses the learned patterns to predict whether it is an apple or banana.

Teorema 6.3 (Critical Assumption)

THe features of training and testing data **must be the same!**. The model can only make predictions based on features it was trained on

6.3 Learning and Generalization

Learning is fundamentally about **generalizing** from the training data. The goal is not simply to memorize the training examples, but to learn patterns that apply to new, unseen data.

However, the failure of a machine learning algorithm is foten caused by a **bad selection of training samples**. The key issue is that we might introduce **unwanted correlations** from which the algorithm derives **wrong conclusions**.

Esempio 6.4 (Recognize object in images)

Consider a model trained to recognize objects in images. If all training images were captured on sunny days, the model might learn to associate sunny weather conditions with the objects. When tested on images taken on cloudy days, the model might fail to recognize the same objects because it never encountered cloudy cnditions during training. The model incorrectly learned that sunny weather is a relevant feature for object recognition.

Quality and diversity of training data are crucial. Training data should be representative of all real-world conditions

7 The complete Machine Learning Recipe

We can now revisit the complete machine learning workflow with all its essential ingredients:

7.1 The five ingredients of Machine Learning

Every machine learning problem requires five fundamental components that work toghether:

1. **Task**: The specific problem we want to solve (e.g., classify birds vs. non-birds in images)
2. **Data**: The collection of examples used to train and evaluate the model. Data quality and quantity are critical for success
3. **Model Hypothesis**: The family of function or models we consider as a potential solutions. This defines the space of possible models the learning algorithm can explore.
4. **Objective Function**: A mathematical function that quantifies how well a model performs on the data. The learning algorithm aims to optimize this function
5. **Learning Algorithm**: The method used to search through the model hypothesis space and find the model that optimizes the objective function.

7.2 The interplay of Ingredients

These five ingredients are interconnected:

- The **task** determines what kind of **data** we need and what **model hypothesis** is appropriate
- The **data** influences which features are available and how we define the **objective function**
- The **model hypothesis** constrains what the **learning algorithm** can discover
- The **objective function** guides the **learning algorithm** toward better models

Understanding and carefully selecting each ingredient is essential for building effective machine learning systems.

8 Model and Hypothesis Space

8.1 What is a Model?

Definizione 8.1 (Model)

A **model** is the implementation of a function $f \in \mathcal{F}_{\text{task}}$ that can be tractably computed.

IN practice, when searching for our function f, we don't look at everything in $\mathcal{F}_{\text{task}}$. Instead, we look at a subset called the **hypothesis space**: $\mathcal{H} \subset \mathcal{F}_{\text{task}}$, which is composed of a set of models (e.g., neural networks, decision trees, linear models).

The learning algorithm seeks a **solution within the hypothesis space**. In other words, a model is a simplified representation of the world that we can actually compute and optimize.

Observation. The choice of hypothesis space is crucial:

- If $\mathcal{H}$ is too small, it may not contain a good solution
- If $\mathcal{H}$ is too large, finding the best model becomes computationally intractable
- The hypothesis space defines the types of patterns the model can learn

8.2 Types of Learning Paradigms

Definizione 8.2 (Three Learning Paradigms)

Machine learning can be categorized into different paradigms based on the type and availability of labeled data:

Supervised learning

In supervised learning, we have labeled data where each input is paired with its corresponding output. This includes:

- **Classification**: Predicting discrete labels
- **Regression**: Predicting continuous values

Unsupervised learning

In unsupervised learning, we only have input data without labels. The goal is to discover structure in the data. That includes:

- **Clustering**: Grouping similar data points
- **Density estimation**: Modeling the probability distribution of data
- **Dimensionality Reduction**: Finding lower-dimensional representations

8.2.1 Supervised Learning

In supervised learning, we have labeled data where each input is paried with its corresponding output.

8.2.2 Unsupervised Learning

In unsupervised learning, we only have input data without labels.

8.2.3 Semi-Supervised Learning

Semi-supervised learning addresses scenarios where we have a large amout of unlabeled data and only some labeled examples. This paradigm has gained significant popularity in recent years to leverage large dataset through self-supervised learning techniques.

The key idea is to provide the model with tasks different from the one to be solved, but that allow the model to learn the data distribution from the unlabeled data. Once the model understands the general structure of the data, it can then specialize on the few labeled examples to solve the target task.

Esempio 8.3 (Training a language)

Training a language model on large amounts of unlabeled text to learn general laguage pattern, then fine-tuning it on a small labeled dataset for sentiment analysis.

9 Model-Based vs. Instance-Based Learning

Machine learning algorithms can be further categorized based on how they make predictions:

9.1 Model-Based (Parametric Learning)

Definizione 9.1 (Parametric Model)

A learning model that summarizes data with a **set of parameters of fixed size** (independent of the number of training examples) is called **parametric model**.

Parametric Model Characteristics

Key characteristics:

- The model learns a fixed set of parameters from the training data
- Once trained, the original training data can be discarded
- No matter how much data you give to a parametric model, it won't change its mind about how many parameters it needs.
- The model makes predictions using only the learned parameters

Examples: Naive Bayes, Linear regression, Logistic Regression, Neural Networks.

In a model-based approach, the algorithm learns a function (e.g., decision boundary) described by parameters. For instance, a linear classifier might learn the curve $\alpha x_1 + \beta x_2 + \gamma$ that separates two classes, where α, β, γ are the parameters.

9.2 Instance-Based (Nonparametric) Learning

Definizione 9.2 (Nonparametric Model)

These algorithms **do not learn parameters** but instead use the data itself to compare a new example through similarity metrics.

Nonparametric Model characteristics

Key characteristics:

- The models stores (some or all of) the training data
- Predictions are made by comparing new examples to stored training examples
- The model's complexity can grow with the amount of training data
- No explicit training phase that produces a fixed set of parameters

Example: K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Random Forest.

In an instance-based approach, when a new example arrives, it is compared with nearby training examples and classified according to a similarity policy. For

example, in KNN, a new point is classified based on the majority class of its k nearest neighbors in the training data.

10 Error Function (Objective Function)

To allow an algorithm to learn, we need to provide it with a metric, a measure that helps it understand the direction of the optimal solution we are seeking. For this reason, **performance measures** are defined to enable the algorithm to learn.

Aspect	Model-based	Instance-Based
Parameters	Fixed number of parameters	No fixed parameters
Training data	Can be discarded after training	Must be retained for predictions
Prediction speed	Fast (onlu uses parameters)	Slower (compares with stored data)
Memory	Low (stores only parameters)	High (stores training examples)
Flexibility	Fixed model complexity	Complexity grows with data

10.1 What is an Error function?

In machine learning, **training a model** means finding a **function** which maps a set of values x to a value y. we can calculate how well a predictive model is doing by comparing the **predicted values** with the **true values** for y.

Definizione 10.1 (Training error vs. Test Error)

- If we apply the model to the data it was trained on, we are calculating the **training error**
- If we calculate the error on data which was **unknown** in the training phase, we are calculating the **test error**

The test error is the most important metric because it tells us how well the model generalizes to new, unseen data.

10.2 Types of error functions

There are different types of error functions (also called loss functions or objective functions), each suited for different task:

- **Mean Absolute Error (MAE)**: Measures the average absolute difference between predictions and true values
- **Mean Squared Error (MSE)**: Measures the average squared difference between predictions and true values
- **Binary Cross-Entropy**: Used for binary classification problems
- **Mean Average Precision**: Used for ranking and information retrieval tasks

The choice of error function depends on the task and the desired properties of the model.

11 Example: Polynomial Fitting

Let's consider a concrete example to understand how models, hypothesis spaces, and error functions work toghether.

11.1 Problem setup

Consider regression problem with the following setup:

- **Data**: $\mathcal{D}_n = \{(x_1, y_1), \ldots, (x_n, y_n)\}$.
- Data generated from $sin(2\pi x)+$ noise
- Training set with $n = 10$ points
- **Goal**: Learn a function (shown as the purple line) that fits the data

The correct solution shold capture the underlying sinusoidal pattern while being robust to the noise in the training data.

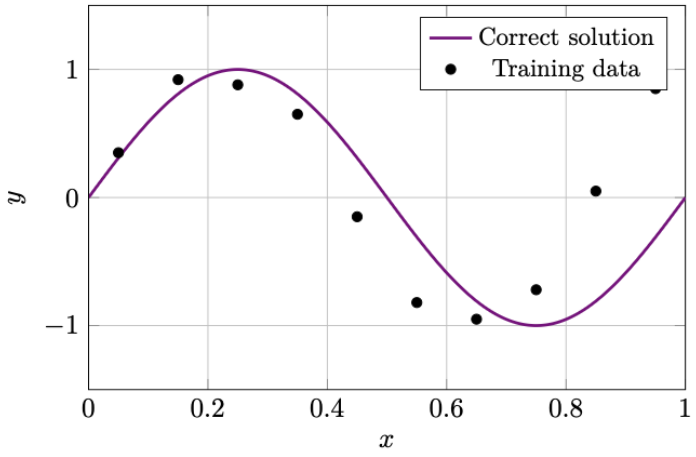


Figura 1: Polynomia Fitting Problem: Data generated from $sin(2\pi x)$ with noise

11.2 Model: Polynomial Functions

Definizione 11.1 (Polynomia Model)

We choose to model the data using polynomia functions:

$$f_w(x) = \sum_{j=0}^{M} w_j x^j$$

where:

- M is the **degree of the polynomial** (a hyperparameter)
- $w = \{w_0, \ldots, w_M\}$ are the **parameters** to be learned
- The **hypotesis space** for a fixed degree M is: $\mathcal{H}_M = \left\{ f_w : w \in \mathbb{R}^{M+1} \right\}$

Different values of M give us different hypothesis spaces:

- $M = 0$: Constant functions (horizontal lines)
- $M = 1$: Linear functions (straight lines)
- $M = 2$: Quadratic functions (parabolas)
- $M = 3$: Cubic functions
- Higher M: More complex curves

11.3 Error function: Mean Squared Error

To measure how well our polynomial fits the data, we use the Mean Squared Error (MSE):

$$E(f; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^{n} [f(x_i) - y_i]^2$$

This error function:

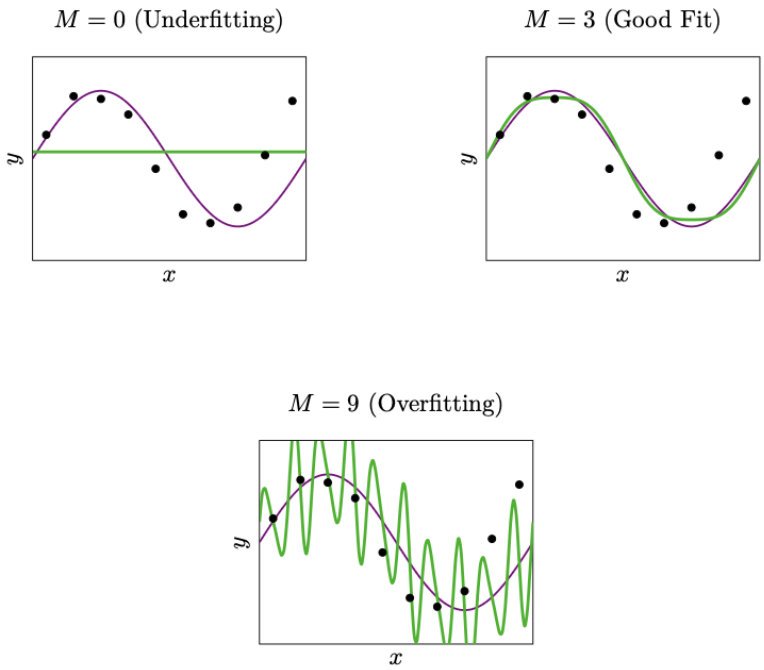


Figura 2: Polynomial fits with different degrees: $M = 0$ (underfitting), $M = 3$ (good fits), $M = 9$ (overfitting). Purple: true function, Green: fitted polynomial, Black dots: training data

- Computes the squared difference between the prediction $f(x_i)$ and the ground-truth label y_i for each point
- Averages these squared differences over all n training points
- Is also called a **pointwise loss** because it measures the error at each individual data point

The learning algorithm's goal is to find the parameters w that minimize this error function:

$$w^* = \arg\min_{w \in \mathcal{R}^{M+1}} E(f_w; \mathcal{D}_n)$$

Choice of Model Comlpexity is Crucial:

- Too small M: Underfitting (cannot captuer pattern)
- Appropriate M: Good fit (captures underlying pattern)
- Too large M: Overfitting (fits noise, poor generalization)

12 Underfitting and Overfitting

Two fundamental problems in ML

Underfitting (too simple) $\leftrightarrow$ Overfitting (too complex)

Two fundamental problems can occur when training machine learning models: underfitting and overfitting. Understanding these concepts is crucial for building models that generalize well.

12.1 Definitions

Definizione 12.1 (Underfitting)

Underfitting is a scenario where a model is unable to capture the relationship between the input (x) and output (y) variables accurately, generating a high error rate on both the training set and unseen data (e.g., testing set).

The model is too simple and has not yet learned from the data. It performs poorly on both training and test data.

Definizione 12.2 (Overfitting)

Overfitting occurs when the model gives accurate predictions for training data (lower training error) but not for testing data (higher testing error).

The model is too sensitive to the training data and has essentially memorized it, including its noise and peculiarities, rather learning the underlying pattern.

12.2 The Bias-Variance Trade-off

The relationship between model complexity, training error can be visualized as follows:

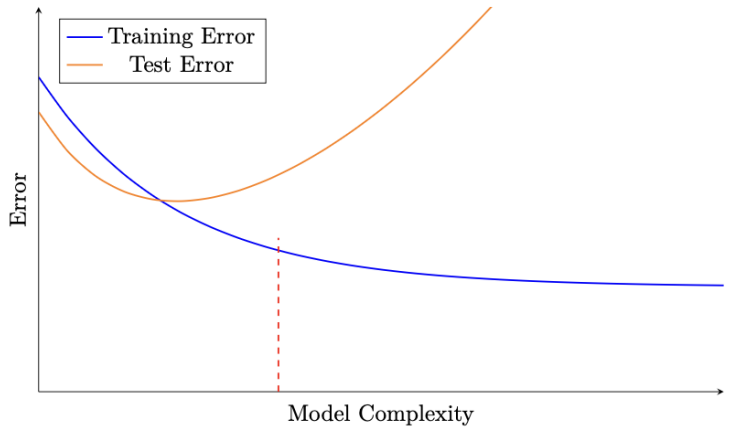


Figura 3: Bias-Variance Trade-off: Training and Test Error vs. Model Complexity

- **Left side of the graph (low complexity)**: High error rate in both training and testing → **Underfitting**
- **Middle Region (optimal complexity)**: Low error rate in both training and testing → **Best Fit**
- **Right side of the graph (high complexity)**: Low error rate in training but high error rate in testing → **Overfitting**

12.3 Diagnostic Matrix

We can dignose the state of our model by examining both training and test errors:

	Low training error	**High training error**
Low Test Error	OK (Goot fit)	Underfitting
High Test Error	Overfitting	Underfitting

12.4 How to handle onverfitting

When the model is too sensitive to training data and overfits, we can apply several strategies:

1. **Try a simpler model**: Use a model with fewer parameteres or lower complexity
2. **Try a less powerful model**: Choose a model architecture with reduced capacity
3. **Increase regularization impact**: Apply techniques that penalize model complexity:
    - Early stopping: stop training before the model overfits
    - L1/L2 regularization: Add penalty terms to the loss function
4. **Use a smaller number of features**:
    - Remove some features that may be causing overfitting
    - Apply feature selection techniques to identify the most relevant features
5. **Get more data**: Increasing the training set size helps the model learn more generalizable patterns

12.5 How to handle unverfitting

When the model has not yet learned from the data and underfits, we can try:

1. **Try more complex model**: Use models with larger number of parameters that can capture more intricate patterns
    - Ensemble learning: Combine multiple models

2. **Less regularization**: Reduce or remove regularization constraints that may be limiting the model's capacity

3. **A larger quantity of features**: Add more features or engineer new features that better capture the underlying relationships (get more features)

12.6 Polynomial Fitting Example Revisited

Let's examine how different polynomial degrees affect the fit:

- $M = 0$ (constant function): Severe underfitting. The horizontal line cannot capture any of the sinusoidal pattern. Both training and validation errors are high
- $M = 1$ (linear function): Still underfitting. A straight line cannot represent the curved pattern. Both errors remain high.
- $M = 3$ (cubic polynomial): Good fit. The model captures the underlying sinusoidal pattern well. Both training and validation errors are low, and the curves are similar
- $M = 9$ (9th-degree polynomial): Overfitting. The model passes through alla training points (very low training error) but oscillates wildly between them. The validation error is high because the model has learned the noise rather than the signal.

The error plot shows:

- **Training error** (blue line): Decresases monotonically as M increases
- **Validation error** (orange line): Decreases initially, reaches a minimum around $M = 3$, then increases again
- The optimal model complexity is where the validation error is minimized

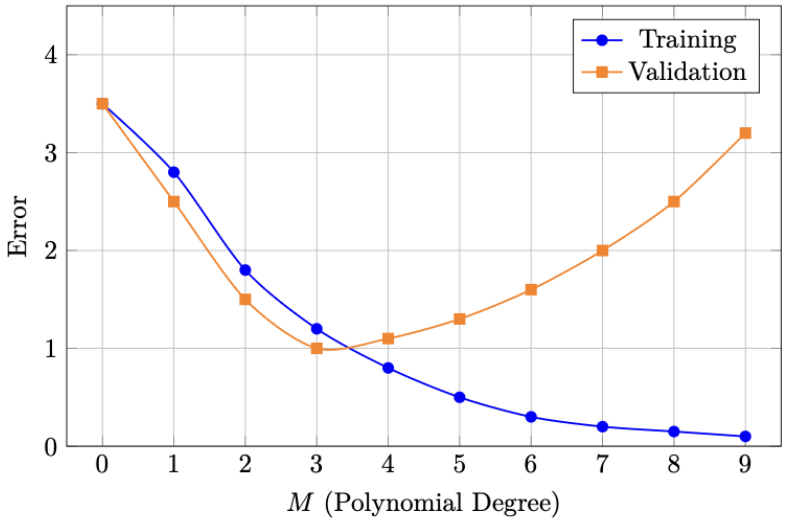
Key Insight

Minimizing training error alone is **not sufficient**
Must monitor validation/test error for generalization

13 Regularization

One of the most important techinques in ML $\rightarrow$ Prevents overfitting and improves generalization

Regularization is one of the most important techniques in machine learning to prevent overfitting and improve model generalization.



H

Figura 4: Training and Validation Error vs. Polynomial Degree M

13.1 What is Regularization?

Definizione 13.1 (Regularization)

Regularization refers to techniques that are used to calibrate machine learning models in order to minimize the adjusted loss function and prevent overfitting or underfitting.

In practice, regularization is a modification of the training error function by appending a term $\omega(f)$ that typically penalizes complex solutions.

13.2 The regularized objective function

The regularized objective function has the following form:

$$E_{reg}(f; \mathcal{D}_n) = E(f; \mathcal{D}_n) + \lambda_n \omega(f)$$

where:

- $E(f; \mathcal{D}_n)$ is the **training error function** (e.g., MSE)
- $\omega(f)$ is the **regularization term** that penalizes model complexity
- λ_n is the **tradeoff hyperparameter** that controls the strngth of regularization

The regularization term acts as a penalty that discourages the model from becoming too complex. By minimizing this combined objective, we find models that fit the data well while remaining relatively simple.

13.3 Effect of Regularization

The regularization term $\omega(f)$ typically measures some notion of model complexity. When we minimize the regularized objective:

- We balance fitting training data (low $E(f; \mathcal{D}_n)$) with keeping the model simple (low $\omega(f)$)
- The hyperparameter λ_n controls this tradeoff
- Higher λ_n means stronger regularization (simpler models)
- Lower λ_n means weaker regularization (more complex models allowed)

13.4 Regularization in Polynomial Fitting

For the polynomial fitting example, we can regularize by penalizing polynomials with large coefficients. The regularized error function becomes:

$$E_{reg}(f_w; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^{n} [f_w(x_i) - y_i]^2 + \frac{\lambda}{n} \parallel w \parallel^2$$

where:

- The fisrt term is the training erorr (MSE)
- $\frac{\lambda}{n} \parallel w \parallel^2 = \frac{\lambda}{n} \sum_{j=0}^{M} w_j^2$ is the regularization term (L2 regularization)
- λ is the tradeoff hyperparameter

13.4.1 Effect of the Regularization Parameter λ

The choice of λ significantly affects the model:

- $\lambda \approx 10^{-18}$ (**very small**): Almost no regularization. The model can have large coefficients and may overfit. The polynomial fits the training data very closely, including noise.
- $\lambda = 1$ (**moderate**): Balanced regularization. The model is penalized for having large coefficients, leading to amoother curve that generalizes better. This often provides the best tradeoff between training and validation error.
- λ **very large**: Strong regularization. The penalty for large coefficients is so high that the model becomes too simple and may underfit. The curve becomes nearly flat.

13.4.2 Regularization Path

The plot of error vs. $ln(\lambda)$ shows:

- **Left side (small λ)**: Low training error, high validation error $\rightarrow$ Overfitting
- **Middle region:** Both errors are low $\rightarrow$ Good fit
- **Right side (large λ)**: Both errors increase $\rightarrow$ Underfitting

The optimal λ is found where the validation error is minimized.

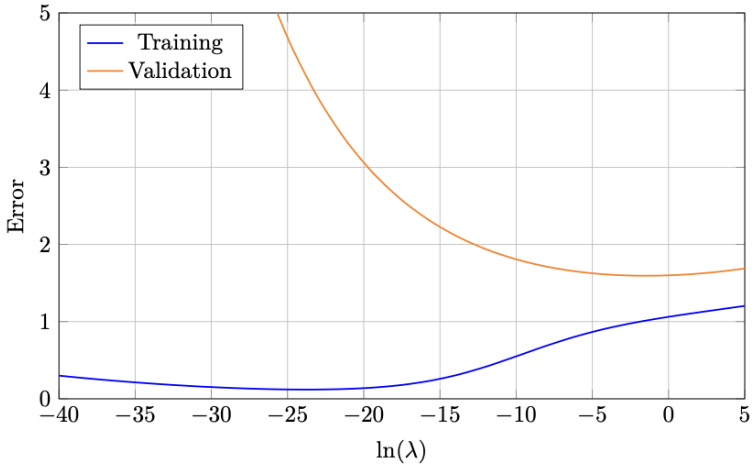


Figura 5: Training and Validation Error vs. $ln(\lambda)$ (Regualarization Strength)

13.5 Generalization and Data Size

An important theoretical result in machine learning relates training erorr to generalization error:

Teorema 13.2 (Generalization with Infinite Data)

As the number of training samples approaches infinity, the training error approzimates the generalization erorr:

$$E(f; \mathcal{D}_n) \to E(f; p_{data}) \text{ as } n \to \infty$$

This means:

- With a small dataset ($n = 15$), even a complex model ($M = 9$) may overfit because there isn't enough data to constrain the parameters
- With a large dataset ($n = 100$), the same complex model can generalize well beacouse the abundant data prevents overfitting
- More data allows us to use more complex models without overfitting

Visual example:

- $n = 15, M = 9$: The 9th-degree polynomial overfits, oscillating widly between the few training points
- $n = 100, M = 9$: With 100 training points, the same 9th-degree polynomial fits smoothly and generalizes well

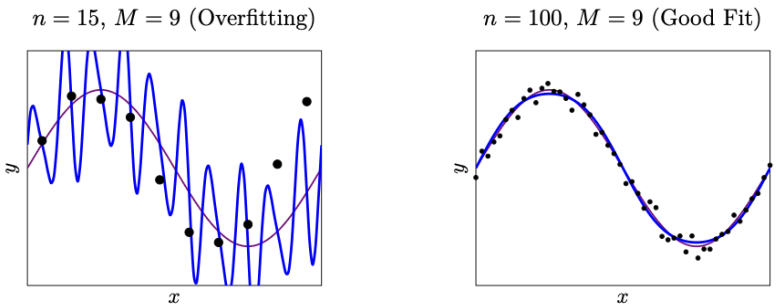


Figura 6: Effect of data size on generalization: With fex data points ($n = 15$), a complex model ($M = 9$) overfits. With many data points ($n = 100$), the same model generalizes well. Purple: true function, Blue: fitted polynomial, Black dots: trining data.

"Get mora data" is often the most effective solution to overfitting. With sufficinet data, even complex models can generalize well.

Parte II
Model Evaluation and Regression Methods

14 Model evaluation for Classification and Regression

14.1 Recap: Train, Validation, and Test Sets

The machine learning workflow involves three distinct datasets, each serving a specific purpose:

14.1.1 The three dataset

1. **Training set**: Used to train the machine learning algorithm
    - The model learns patterns and relationships from this data
    - Parameters are optimized to minimize training error
2. **Validation set**: Used during the learning phase
    - Also called the *mini test set*
    - Helps select the better performing algorithm among alternatives
    - Used to tune hyperparameters of the model

- Provides feedback to adjust the model before final testing

3. **Test set**: User for final model evaluation
    - Provides an unbiased assessment of model perfromance
    - The model has never seen this data during training or validation
    - Tells us how well the model will perform on new, real-world data

14.2 The importance of the validation set

The validation set plays crucial role in the machine learning pipeline and must kept **separate from the training set**.

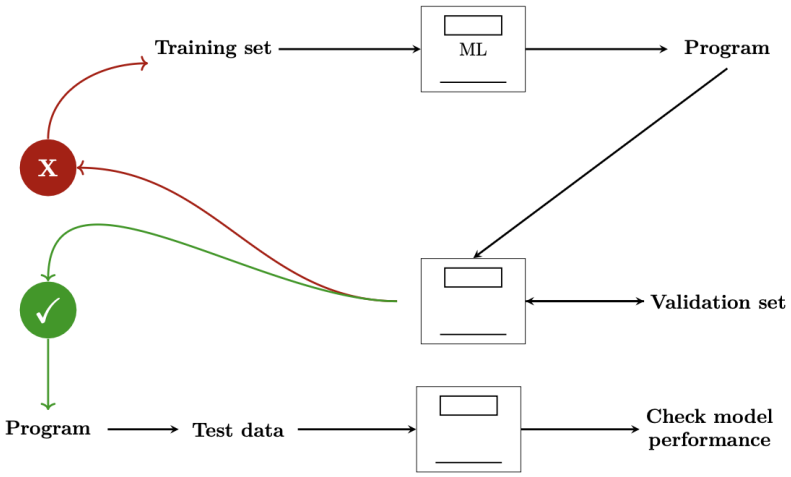


Figura 7: Machine learning workflow. Training set trains the model, validation set evaluates and provides feedback (thumbs down = retain, thumbs up = proceed to testing), test set checks final perfromance

14.2.1 Why separate the validation set?

The validation set is used for two main purposes:

1. **Algorithm Selection**: To pick the better performing algorithm
    - Compare different model architectures (e.g., decision trees vs. neural networks)
    - Select the model that performs best on unseen data
2. **Hyperparameter Tuning**: To decide the hyperparameters of an algorithm
    - Hyperparameters are configuration setting that are not learned from data
    - Examples: learning rate, regularization strength (λ), number of layers, tree depth

- The validation set helps find the optimal hyperparameter values

Note. **ATTENTION**: Splitting the datasets into training and validation sets can be done randomly to avoid bias. However, tehre are some **specific rules** to apply when performing this split to ensure reliable model evaluation.

14.3 Specific Rules for Dataset Splitting

When splitting data into training, validation, and testing sets, you can have conflicting priorities. The key is to balance these priorities appropriately.

14.3.1 Priority 1: Accurate Error Estimation

Goal: Estimate future error (i.e., validation/testing error) as accurately as possible.

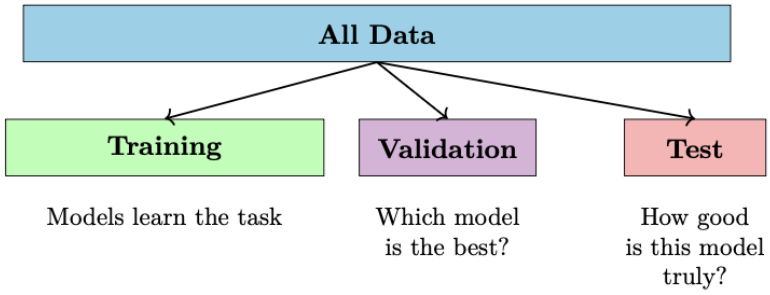


Figura 8: Dataset splitting: Training for learning, Validation for model selection, Test for final evaluation

How to achieve this: By making the **validation set as big as possible**.

(High confidence interval)

- A larger validation set provides more reliable error estimates
- Reduces variance in performance metrics
- Gives higher confidence that the measured perfromance reflects true generalization

14.3.2 Priority 2: Accurate Classifier Learning

Goal: Learn classifier as accurately as possible.

How to achieve this: By making the **training set as big as possible**

(Better estimates, maybe better generalization)

- More training data allows the model to learn more robust patterns

- Reduces overfitting by providing diverse examples
- Generally leads to better generalization on unseen data

14.3.3 The critical Rule

CRITICAL REQUIREMENT → Training and validation/testing instance **CANNOT OVERLAP !!!**

- If the same data appears in both training and validation/test sets, the evaluation is invalid
- The model would be tested on data i has already seen during training
- This lead to overlay optimistic performance estimates that don't reflect real-world performance
- Data leakage between sets is one of the most common mistakes in machine learning

Observation. The split between training and validation/test sets represents a fundamental trade-off:

- Larger training set → Better model learning, but less reliable error estimation
- Larger validation/test set → More reliable error estimation, but potentially worse model learning

Common splits include 70-15-15, 80-10-10, or 60-20-20 (train-validation-test), depending on the total dataset size and specific requirements.

15 Cross Validation

15.1 When do we need Cross Validation?

In cases where we don't have enough data to randomly split between training (e.g., 60% of the totale data), validation (e.g., 20% of the data), and testing (e.g., 20% of the totale data), we need to use **cross-validataion**.

Definizione 15.1 (Cross Validation)

Cross-Validation is a resampling technique that involves reusing a portion of the training set itself to validate performance by retraining not just one, but N models.

15.2 Key characteristics of Cross Validation

- **Multiple models**: Cross-Validation involves training N models instead of just one
- **Computional cost**: Performing cross-validation requires N training sessiones and can be more computationally demanding
- **Better use of limited data**: When data is scarce, cross-validation, allows us to use the available data more efficently for both training and validation

15.3 Fundamental Rules of Cross Validation

Cross validation must respect certain constraints to ensure valid model evaluation:

1. **No overlap**: Training and validation cannot overlap, but $n_{train} + n_{validation}$ = constant
2. **Each instance used once per fold**: At each fold (step), you use each instance only in one set, so there is no overlapping within signle fold
3. **Every sample participates**: Every sample is in both training and testing but not at the same time
    - This reduces the changs of getting a biased training set
    - Ensures all data contributes to both training and validation across different folds

15.4 K-Fold Cross Validation

K-fold cross validation is the most common form of cross validation, where the dataset is divided into k equal parts (folds).

15.4.1 How K-Fold Cross Validation works

In the case of k-fold cross-validation:

1. We split out dataset into k groups (folds)
2. We perform training k times on $(N/k)(k-1)$ data points
3. We measure performance on the last N/k data points (the validation fold)
4. Our cross-validation performance will be the **average of the perfromance of the k tests**

Note. **Important constraint**: Training set and validation set cannot overlap, but the sum of training and validation data is constant (the totale dataset size).

15.4.2 Visual Representation of K-Fold Cross Validation

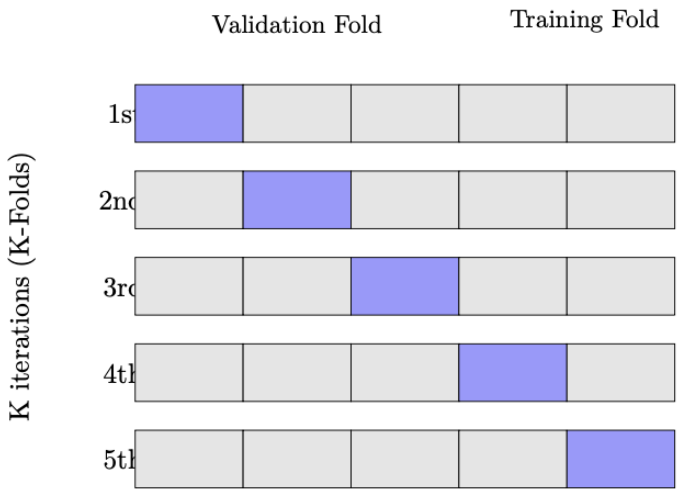


Figura 9: K-Fold Cross Validation: Each row represents on iteration where a different fold serves as validation (blue) while the rest serve as training (gray)

15.5 Example: 5-Fold Cross Validation

Let's examine a concrete example with 5 folds:

- **Step 1**: Randomly split the data into 5 folds
- **Step 2**: Test on each fold while training on 4 other folds (80% train, 20% test)
- **Step 3**: Average the results over 5 folds



Figura 10: 5-Fold Cross Validation Example: Each fold serves as validation once, and the final perfromance is the average of all 5 iterations

15.5.1 The process in Detail

For each of the 5 iterations:

1. **Iteration 1**: Use fold 1 as validation, folds 2-5 as training → Get Performance 1
2. **Iteration 2**: Use fold 2 as validation, folds 1,3-5 as training → Get Performance 2
3. **Iteration 3**: Use fold 3 as validation, folds 1-2,4-5 as training → Get Performance 3
4. **Iteration 4**: Use fold 4 as validation, folds 1-3,5 as training → Get Performance 4
5. **Iteration 5**: Use fold 5 as validation, folds 1-4 as training → Get Performance 5

Final Cross-Validation Performance:

$$\text{CV Performance } = \frac{1}{5}\sum_{i=1}^{5} \text{Performance}_i$$

15.6 Advantages of Cross Validation

- **Better use of limited data**: Every data point is used for both training and validation
- **More reliable performance estimates**: Averaging over multiple folds reduces vairance in the performance estimate
- **Reduces bias**: Every sample participates in validation, reducing the risk of biased validation set
- **Detects overfitting**: If performance varies significantly across folds, it may indicate overfitting or data quality issues

15.7 Disadvantages of Cross Validation

- **Computationally expensive**: Requires training k models instead of one
- **Time-consuming**: Training time is multiplied by the number of folds
- **Not suitable for very large datasets**: When a data is abundant, a simple train-validation-test split is more efficient

Observation. The choice of k (number of folds) involves a tradeoff:

- **Larger** k: More accurate performance estimate, but more computationally expensive
- **Smaller**: k: Faster computation, but less reliable performance estimate

- Common choices: $k = 5$ or $k = 10$.
- Extreme case: $k = N$ (Leave-One-Out Cross Validation) uses each single sample as validation once

16 Leave-One-Out Cross Validation

16.1 What is Leave-One-Out Cross Validation?

Leave-One-Out Cross Validation (LOOCV) is an extreme case of k-fold cross validation that is particularly useful when we have very few data points.

Definizione 16.1 (Leave-One-Out Cross Validation)

LOOCV is n-fold cross validation where n equals the total number of samples. In each iteration, we train on all $(n-1)$ samples and test on exactly 1 instance.

16.2 How LOOCV Works

The process involves:

1. Take the entire dataset of n samples
2. For each sample i (where $i = 1, 2, \ldots, n$):
   - Use sample i as the validation set (1 sample)
   - Use all other samples $(n-1)$ as the training set
   - Train the model and evaluate on the single validation sample
3. Repeat this process n times (once for each sample)
4. Average the performance across all n expriments

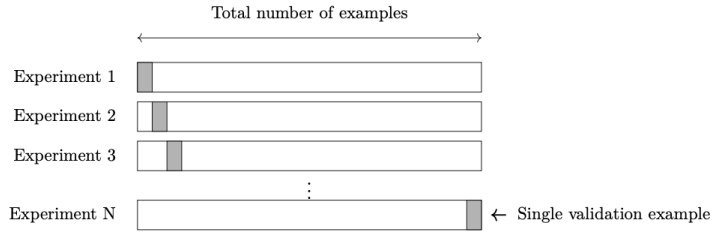


Figura 11: Leave-One-Out Cross Validation: Each experiment uses exactly one sample for validation (gray) and all others for training (white)

16.3 Pros and Cons of LOOCV

16.3.1 Advantages

- **Best possible classifier**: The model is learned from $n-1$ training examples, which is the maximum possible training data
- **Maximizes training data**: Each model uses almost all available data for training
- **Unbiased estimate**: Provides a nearly unbiased estimate of model performance
- **Deterministic**: No randomness in the splitting process (every sample is used exactly once as validation)

16.3.2 Disadvantages

- **High computational cost**: Must re-learn everything for n times, where n is the total number of samples
- **Time-consuming**: For large datasets, LOOCV becomes impratical
- **High variance**: Performance estimates can have high variance, especially with small datasets

Note. LOOCV is most appropriate when:

- The dataset is very small (e.g., $n < 50$)
- Computational resources are available
- You need the most accurate performance estimate possible

For larger datasets, standard k-fold cross validation (with $k = 5$ or $k = 10$) is typically preferred due to its better balance between computation cost and performance estimation accuracy.

17 Stratification in Cross Validation

17.1 The problem: Imbalanced Classes

In many real-world datasets, class labels are not evenly distributed. For example, in a medica diagnosis dataset, healthy patients might vastly outnumber sick patients. When performing cross validation on such imbalanced datasets, random splitting can lead to folds with very different calss distribution.

17.2 What is Stratification?

Definizione 17.1 (Stratification)

Stratification is a technique to keep class labels **balanced across training and validation sets** during cross validation. Instead of randomly diving the dataset, we ensure that each fold maintains approximately the same proportion

of samples for each class as the complete dataset.

17.3 How Stratified K-Fold Works

The stratification process works as follows:

1. **Divide by class**: Instead of taking the dataset and dividing it randomly into K parts, take the dataset and divide it into individual classes
2. **Split each class**: Then for each class, divide the instances into K parts
3. **Assemble folds**: Assemble the i-th part from alla classes to make the i-th fold
4. **Important**: It still random! The splitting within each class is random, but the proportions are maintained

17.4 Example: Stratified 5-Fold Cross Validation

Let's consider a concrete scenario with an imbalanced dataset containing 300 samples, divided into three classes (A, B, and C), where Class A has more samples than the other two.

17.4.1 Imbalanced Dataset

- **Class A**: 200 samples (66.7%).
- **Class B**: 50 samples (16.7%).
- **Class C**: 50 samples (16.7%).
- **Total**: 300 samples

17.4.2 Split the Dataset into Five Folds with Stratification

Each fold will have 60 samples total, maintaining the class proportions:

- **Fold 1**: 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 2**: 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 3**: 60 samples (Class A: 40, Class B: 10, Class C: 10)

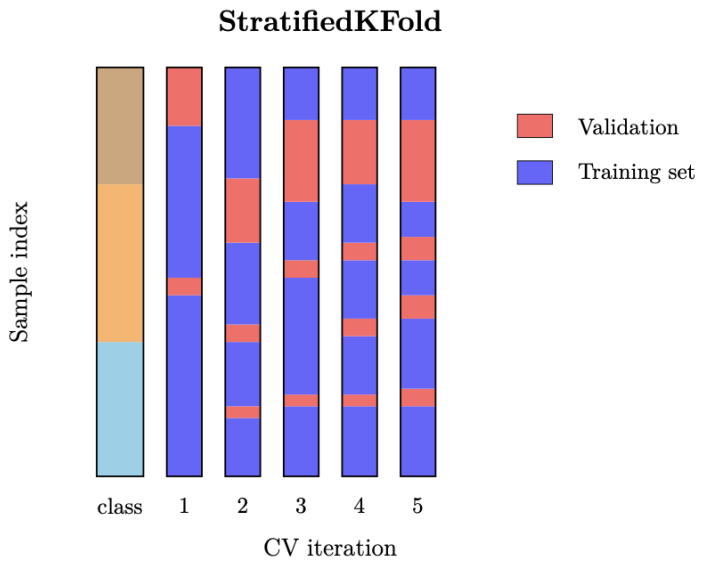


Figura 12: Stratified K-Fold: Each fold maintains the same class proportions across all folds

- **Fold 4**: 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 5**: 60 samples (Class A: 40, Class B: 10, Class C: 10)

Notice that each fold maintains the same $66.7\% - 16.7\% - 16.7\%$ class distribution as the original dataset.

17.4.3 Train and Validate

The cross-validations proceeds as follows:

1. **Iteration 1**: Train on Folds 1, 2, 3, and 4, Validate on Fold 5
2. **Iteration 2**: Train on Folds 2, 3, 4, and 5, Validate on Fold 1
3. **Iteration 3**: Train on Folds 1, 3, 4, and 5, Validate on Fold 2
4. **Iteration 4**: Train on Folds 1, 2, 4, and 5, Validate on Fold 3
5. **Iteration 5**: Train on Folds 1, 2, 3, and 5, Validate on Fold 4

17.4.4 Average the performance Metrics

- Calculate performance metrics for each iteration
- Average these metrics to evaluate the model's performance robustly

17.5 Why Stratification Matters

- **Prevents biased folds**: Without stratification, some folds might have very few (or no) examples of minority classes
- **Consistent evaluation**: Each fold represents the overall dataset distribution, leading to more reliable performance estimates
- **Better for imbalanced datasets**: Particularly important when dealing with class imbalance
- **Reduces variance**: Performance estimates have lower variance across folds

Note. **When to use stratification**:

- Always use stratification for classification problems with imabalanced classes
- It's generally a good practice even for balanced datasets
- Most machine learning librearies (e.g., scikit-learn) provide stratified cross-validation functions
- For regression problems, stratification is not applicable (since there are no discrete classes)

Observation. Stratification ensures that:

- Each fold is a good representative of the whole dataset
- Minority classes are adequately represented in both training and validation sets
- Performance metrics are more stable and reliable across folds
- The model ses a balanced distribution of classes during each training iteration

18 Evaluation Measures

Once we have trainend out models and performed cross-validation, we need to evaluate their performance. Different tasks require different evaluation metrics.

18.1 Why do we need Evaluation Measures?

Evaluation measures serve two primary purpose:

1. **To decide if our model is performing well**: Assess wether the model meets the requirements for the task
2. **To decide out of many models, which one performs better than the other**: Compare different models and select the best one for deployement

18.2 Evaluation Measures by Task Type

Different nachine learning tasks require different evaluation approaches:

18.2.1 Classification

Question: How often does our model classify something right/wrong?

Classification metrics measure the correctness of predictions, focusing on whether the predicted class matches the actual class.

18.2.2 Regression

Question: How close our model to what we are trying to predict?

Regression metrics measure the distance or error between predicted continuos values and avtual values.

18.2.3 Clustering

Question: How well does our model describe our data?

Clustering evaluation is generally very hard because there's often no ground truth to compare against. Metrics must assess the quality of discovered patterns.

19 Evaluation Measures of Classification

19.1 Types of Classification Problems

A classification model takes a set of features as input and produces one or more classes as output.

Definizione 19.1 (Classification Type)

Classification can be categorized into different types based on the number of classes:

- **Binary classification**: In the case of two classes (0/1) or one vs rest (e.g., apples vs other fruits)
- **Multi-class Classification**: In the case having N classes to choose from (e.g., cat, dog, wolf, etc.)
- **Multi-label Classification**: A problem where an example can belong to multiple calsses simultaneously

19.2 The confusion matrix

The confusion matrix is a fundamental tool for evaluating binary classification models. It provides a complete picture of how the model perfrom across different types of predictions

19.2.1 Confusion Matrix for Binary classification

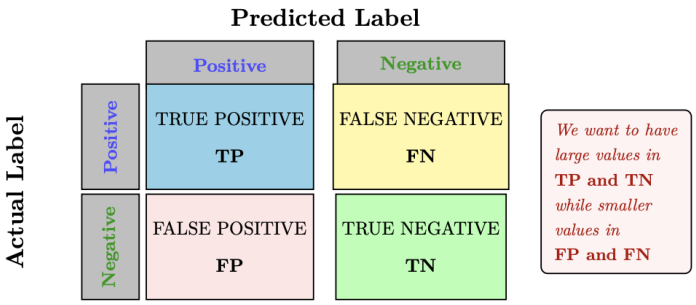


Figura 13: Confusion Matrix for Binary Classification

19.2.2 Understanding the Confusion Matrix

The confusion matrix has four components:

- **True Positive (TP)**: The model correctly predicted the positive class
  - Actual label: Positive
  - Predicted label: Positive
  - Good prediction!
- **True Negative (TN)**: The model correctly predicted the negative class
  - Actual label: Negative
  - Predicted label: Negative
  - Good Prediction!
- **False Positive (FP)**: The model incorrectly predicted positive (Type I error)
  - Actual label: Negative
  - Prediction label: Positive
  - Bad prediction! (False alarm)
- **False Negative (FN)**: The model incorrectly predicted negative (Type II error)
  - Actual label: Positive
  - Prediction label: Negative
  - Bad prediction! (Missed detection)

Note. **Goal**: We want to maximize TP and TN (correct predictions) while miniimzing FP and FN (incorrect predictions).

19.3 Classification Error and Accuracy

From the confusion matrix, we can derive several important metrics.

19.3.1 Classification error

Definizione 19.2 (Classification Error)

The classification error (also called misclassification rate) measures the proportion of incorrect predictions:

$$\text{Classification Error } = \frac{FP + FN}{TP + TN + FP + FN}$$

This represent the fraction of all predictions that were wrong

19.3.2 Accuracy

Definizione 19.3 (Accuracy)

Accuracy measures the proportion of correct predictions:

$$\text{Accuracy } = 1 - \text{Error} = \frac{TP + TN}{TP + TN + FP + FN}$$

This represent the fraction of all predictions that were correct

19.4 The problem with Accuracy: Imbalanced Classes

While accuracy is a basic measure of "goodness" of a classifier, it can be very misleading when dealing with imbalanced classes.

ATTENTION! $\rightarrow$ Accuracy is **very misleading** if we have **imbalanced classes**

19.4.1 Example: The imablanced Dataset Problem

Consider a dataset with severe class imbalance:

- 90 samples for class "NO"
- 10 samples for class "YES"

Native strategy: A trivial classifier that predicts "NO" for every sample would achieve:

$$\text{Accuracy } = \frac{90}{100} = 90\%$$

The problem: Despite having 90% accuracy, this model is completely useless! It cannot classify any data belonging to the "YES" class. The model has learned nothing meaningful about the data.

Observation. The example illusatrate why accuracy alone is insufficient for evaluating classifiers, especially with imablanced dataset. We need additional metrics that provide a more nuanced view of model performance across all classes.

Note. When dealing with imbalanced dataset, always consider:

- Using stratified cross-validation
- Examining the confusion matrix in detail
- Using additional metrics beyond accuracy (precision, recall, F1-score)
- Considering class-specific performance
- Using techniques like oversampling, undersampling, or class weights

19.5 Misses and False Alarms

Beyond accuracy, we need more specific metrics to understand model performance. These metrics focus on different aspects of classification errors.

19.5.1 False Alarm Rate (False Positive Rate)

Definizione 19.4 (False Alarm Rate = False Positive Rate)

$$\frac{FP}{FP+TN}$$

This represents the *percentage of negative samples we misclassified as positive*

19.5.2 Miss Rate (False Negative Rate)

Definizione 19.5 (Miss Rate = True Positive Rate)

$$\frac{TP}{TP+FN}$$

This represent the *percentage of positive samples we misclassified as negative*

19.5.3 Recall (True Positive Rate)

Definizione 19.6 (Recall = True Positive Rate)

$$\frac{TP}{TP+FN}$$

This represent the *percentage of positive samples we classified correctly*

It also know as **Sensitivity** and can be expressed as: Recall = 1 - Miss Rate

Observation. Recall answers the question: "Of all the actual positive cases, how many did we identity?"

19.5.4 Precision

Definizione 19.7 (Precision)

$$\frac{TP}{TP+FP}$$

This represent the *percentage positive out of what we predicted was positive.*

Observation. Precision answer the question: "Of all the cases we predicted as positive, how many were actually positive?"

19.6 Cost of the Task

In pratice, we typically do not use evaluation mertics like accuracy, recall, or precision alone. Instead, we declare a combination of them together based on the specific requirements of the task

19.6.1 The need for a single evaluation measure

- In oreder to optimize a learner automatically (i.e., during training), we need a **single evaluation measure**
- How do we decide that single metric?
  - **Domain specific!** - It depends on the task

19.6.2 weighting Errors by Cost

Depending on the **cost of the task**, we can decide whether we take more about having:

- *Less false positives*, or
- *Less false negatives*

This is achieved through wighting them:

$$\text{Cost} = C_{FP} \times \text{FP} \ + C_{FN} \times \text{FN}$$

where C_{FP} and C_{FN} are the costs associated with false positive and false negatives, respectively.

Esempio 19.8 (Medical Diagnosis (Cancer detection))

- False Negative (missing a cancer diagnosis): Very high cost - patient doesn't receive treatment
- False Positive (false alarm): Lower cost - patient undergoes additional tests
- Therefore: $C_{FN} \gg C_{FP}$, we prioritize high recall

Esempio 19.9 (Spam Email Filter)

- False Positive (marking important email as spam): High cost - user misses important messages
- False Negative (spam gets through): Lower cost - minore annoyance
- Therefore: $C_{FP} > C_{FN}$ we prioritize high precision

19.7 F-Measure (F1 Score)

The F-Measure combines precision and recall into a single metric, providing a balance between the two.

Definizione 19.10 (F1)

The **F1 Score** is the harmonic mean of precision and recall:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Harmonic mean of precision and recall, one of the most frequently used metrics in machine learning

19.7.1 Properties of F1 Score

- $(+)$ If you do some mathematics, you will see that the F1 measure is sort of an accuracy without TN
- $(+)$ Used frequently in information retrieval system
- $(+)$ Provides a singke number that balances precision and recall
- **Range**: $F_1 \in [0, 1]$, where 1 is perfect and 0 is the worst

Note. The harmonic mean (used in F1) is always less than or equal to the arithmetic mean. This means F1 score give more weight to the lower value between precision and recall, ensuring both metrics need to be reasonably high for a good F1 score.

19.7.2 Generalized F-Measure

The F1 score can be generalized to F_β score, which allows adjusting the relative importance of precision and recall:

$$F_\beta = (1 + \beta^2) \times \frac{\text{Precision} \times \text{Recall}}{\beta^2 \times \text{Precision} + \text{Recall}}$$

where:

- $\beta < 1$: Emphasizes precision
- $\beta > 1$: Emphasizes recall
- $\beta = 1$: Equal weight (standard F1 score)

19.8 Multiclass Classification - Evaluation

For multiclass classification problems (more than two classes), we need to extend our evaluation metrics.

19.8.1 Multiclass confusion matrix

- n_{ij} is the number of examples with actual label (true label) y_i and predicted label y_j.
- The **main diagonal** contains true positive for each class
- The **sum of off-diagonal elements along a column** is the number of false positives for the column label
- The **sum of off-diagonal elements along a row** is the number of false negatives for the row label

19.8.2 Multiclass Metrics

For each class i, we can compute:

$$FP_i = \sum_{j \neq i} n_{ij} \quad FN_i = \sum_{j \neq i} n_{ij}$$

$$\text{Precision}_i = \frac{n_{ii}}{n_{ii} + FP_i} \quad \text{Recall}_i = \frac{n_{ii}}{n_{ii} + FN_i}$$

$$\text{Multiclass Accuracy (MAcc)} = \frac{\sum_i n_{ii}}{\sum_i \sum_j n_{ij}}$$

19.8.3 Macro vs. Micro averaging

When reporting overall metrics for multiclass problem, we have two main approaches:

- **Macro-averaging**: Compute metric for each class independently, then take the average

  $$\text{Macro-Precision} = \frac{1}{C} \sum_{i=1}^{C} \text{Precision}_i$$

  This gives equal weight to each class regardless of size
- **Micro-averaging**: Aggregate the contributions of all classes to compute the average metric

  $$\text{Micro-Precision} = \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i)}$$

  This gives equal weight to each sample

19.9 ROC Curve - Receiver Operating Characteristic

The ROC curve is a powerful tool for evaluating and comparing binary classifiers, especially when the decision threshold can be varied.

19.9.1 What is the ROC curve?

The **ROC curve** is a graphical plot that illustrates the diagnostic ability of a binary classifier as its discrimination threshold is varied.

- **X-axis**: False Positive Rate (FPR) = $\frac{FP}{FP+TN}$.
- **Y-axis**: True Positive Rate (TPR) = Recall = $\frac{TP}{TP+FN}$.

19.9.2 Uses of ROC Curves

The ROC curve allows us to:

- **Compare different classifiers**: Different models can be plotted on the same ROC space
- **Estimate the false positive rate / negative rate depending on the threshold** that we set to classify
- Visualize the trade-off between sensitivity (recall) and specificity

19.9.3 Understanding the ROC space

- **Perfect classifier**: A point at $(0, 1) - 100\%$TPR $, 0\%$ FPR.
- **Random classifier**: Lies on the diagonal line from $(0, 0)$ to $(1, 1)$.
- **Better classifier**: Curves that bow toward the upper left corner
- **Worse classifier**: Curves below the diagonal (worse than random guessing)

19.9.4 Area under the ROC Curve (AUC)

Definizione 19.11 (Area under ROC Curve)

The **Area under the ROC Curve (AUC)** is a single scalar value that summarize the performance of a classifier across all possible thesholds

$$AUC \in [0, 1]$$

Goal: Maximize the Area under the ROC Curve (AUC)

19.9.5 Interpreting AUC Values

- **AUC = 1.0**: Perfect classifier
- **AUC = 0.9 - 1.0**: Excellent
- **AUC = 0.8 - 0.9**: Good
- **AUC = 0.7 - 0.8**: Fair
- **AUC = 0.6 - 0.7**: Poor
- **AUC = 0.5**: No better than random guessing

- **AUC < 0.5**: Worse than random (predictions are inverted)

Observation: The AUC can be interpreted as the probability that the classifier will rank a randomly chosen positive instance higher that a randomly chosen negative instance

19.9.6 Advantages of ROC and AUC

- **Threshold-independent**: Evaluates the model across all possible threshold
- **Class imbalance robust**: Unlike accuracy, AUC is less sensitive to class imbalance
- **Single metric**: Provides a single value for model comparison
- **Visual interpretation**: The ROC curve provides intuitive visualization of classifier performance

20 Regression Evaluation Metrics

While classification tasks allow us to count correct and incorrect predictions, regression tasks require different evaluation approaches since we're predicting continuos values.

20.1 The Fundamental Difference

- **Classification**: We can count how often we are correct or wrong in out predictions
- **Regression**: We cannot do that counting!
  - Predicting y_i (not discrete, continuos value) from inputs x_i.
  - The question is not "*how many times your method is wrong*" but "*how much your method is wrong with respect to the groundtruth-labels*"

Key Difference:

$$\text{Classification} : \text{Count errors} \leftrightarrow \text{Regression} : \text{Measure error magnitude}$$

20.2 Mean Squared Error (MSE)

The mean squared error is one of the most commonly used metrics for regression task.

Definizione 20.1 (Mean Squared Error)

$$MSE(f) = \frac{1}{N}\sum_{i=1}^{N}(f(x_i) - y_i)^2$$

where:

- f is the model that takes a feature vector x as input and generates a

prediction $f(x_i)$.

- i, ranging from 1 to N, denotes the index of sample the dataset
- y_i is the ground truth (actual value)

20.2.1 Properties of MSE

- The sum of predictions minus real values over N indicates the average
- **Squaring** removes the negative sign and gives more weight to larger differences
- **The lower the MSE, the better**
- MSE is **influenced by outliers**, i.e., data values that are abnormally distant from the true regression line
- MSE is **differentiable**, which makes it suitable for gradient-based optimization algorithms

Observation: The squaring operation in MSE has two important effects:

1. It ensure all errors are positive (no cancellation between positive and negative errors)
2. It penalizes large errors more heavily that small errors (quadratic penalty)

20.3 Mean absolute error (MAE)

The Mean Absolute Error provides an alternative way to measure regression performance

Definizione 20.2 (Mean Absolute Error)

$$MAE(f) = \frac{1}{N}\sum_{i=1}^{N} \mid f(x_i) - y_i \mid$$

20.3.1 Properties of MAE

- MAE is similar to MSE in that it returns absolute values of the residual $f(x_i) - y_i$ without squaring
- It does not consider the direction of the error, which means we will not know whether negative or positive errors weigh more on the overall average
- **MAE is more robust to outliers** precisely due to the absence of squaring of distant forecast error values
- However, MAE is **not differentiable** at zero. It means that we cannot take the derivative of it, and taking derivatives is important if you want to build an algorithm that minimizes the function. Because of this, it used a lot less than MSE

MSE vs MAE Comparison

- **MSE**: Differentiable, penalizes large errors more, sensitive to outliers
- **MAE**: Not differentiable at zero, treats all errors equally, robust to outliers

20.4 Choosing Between MSE and MAE

The choice between MSE and MAE depends on the specific requirements of your regression task:

Metric	Use When	Avoid When
MSE	• Large errors are particularly undesirable • You need gradient-based optimization • Outliers should be heavily penalized	• Dataset contains many outliers • All errors should be treated equally
MAE	• Outliers are present in the data • All errors should be weighted equally • You want a more interpretable metric	• You need differentiability for optimization • Large errors should be penalized more

20.5 Other Regression Metrics

Beyond MSE and MAE, there are other useful metrics for evaluating regression models:

20.5.1 Root Mean Squared Error (RMSE)

$$RMSE(F) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(f(x_i) - y_i)^2}$$

RMSE is simply the square root of MSE. It has the advange of being in the same units as the target variable, making it more interpretable.

20.5.2 R-Squared (R^2) Score

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - f(x_i))^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the observed values. R^2 represents the proportion of variance in the dependent variable that is predictable from the independent variables. It ranges from 0 to 1, with 1 indicating perfect predition.

Note. In pratice, it's often beneficial to report multiple metrics to get comprehensive view of model performance. For example, reporting both MSE (or RMSE) and MAE can provide insights into both the typical error magnitude and the presence of outliers.